

Journal of the Operational Research Society

ISSN: 0160-5682 (Print) 1476-9360 (Online) Journal homepage: www.tandfonline.com/journals/tjor20

Solving arc routing problems with turn penalties

J Clossey, G Laporte & P Soriano

To cite this article: J Clossey, G Laporte & P Soriano (2001) Solving arc routing problems with turn penalties, Journal of the Operational Research Society, 52:4, 433-439, DOI: 10.1057/palgrave.jors.2601052

To link to this article: https://doi.org/10.1057/palgrave.jors.2601052

Published online: 21 Dec 2017.

Submit your article to this journal

Article views: 77

View related articles

Citing articles: 1 View citing articles

Full Terms & Conditions of access and use can be found at
https://www.tandfonline.com/action/journalInformation?journalCode=tjor20

Journal of the Operational Research Society (2001) 52, 433–439 ©2001 Operational Research Society Ltd. All rights reserved. 0160-5682/01 $15.00

www.palgrave.com/jors

Solving arc routing problems with turn penalties

J Clossey, G Laporte* and P Soriano

GERAD and École des Hautes Études Commerciales, Montréal, Canada

In several arc routing problems, it is necessary to take turn penalties into account when designing a solution. Traditionally, this is done through a transformation of the arc routing problem into an equivalent vertex routing problem. In this paper it is shown that a more direct approach, not resorting to such a transformation, may be more efficient.

Keywords: arc routing problems; turn penalties; travelling salesman problem; Chinese postman problem; heuristics

Introduction

Arc routing problems arise in situations where it is necessary to determine a least cost traversal of some arcs or edges of a graph. Several applications are encountered in the areas of snow removal,[1] street cleaning and road gritting,[2–5] garbage collection,[6,7] mail delivery,[8,9] meter reading,[9,10] etc. For recent overviews, see Eiselt, Gendreau and Laporte[11] and Assad and Golden.[12] In some contexts the cost of a route is not only determined by its length, but also by the types of turns made at intersections. For example, left turns are usually more complex and thus more costly than right turns, and U-turns, when permitted, tend to be quite prohibitive. It therefore makes sense to assign a cost to each type of turn and account for it in the model and solution procedure. Examples of applications where turning costs are explicitly taken into account are provided in Bodin and Kursh,[2] McBride,[6] Roy and Rousseau,[9] Gendreau *et al*[13] and Lee *et al*.[14]

Several ways of modelling turn penalties have been proposed. All make use at some stage of a graph transformation and some solve a *Travelling Salesman Problem* (TSP) associated with the transformed graph. Our aim is to describe a direct approach not requiring a graph transformation and to provide computational comparisons with the standard graph transformation methodology.

The remainder of this paper is organized as follows. The next section contains some background material on the solution of the *Chinese Postman Problem* (CPP) and on the determination of shortest paths with turn penalties. Transformation procedures for the CPP with turn penalties are described, the directed and undirected cases being treated separately. A direct method that does not resort to a graph transformation is then proposed. This is followed by comparative computational results and by the conclusion.

*Correspondence: G Laporte, GERAD and École des Hautes Études Commerciales, 3000 chemin de la Côte-Sainte-Catherine, Montréal, Canada H3T 2A7.
E-mail: gilbert@crt.umontreal.ca

Background material

The algorithm proposed in this paper is based on the solution of the CPP. However, since some of the methods to which the algorithm is compared compute shortest paths with turn penalties, it is important to understand their working. Descriptions of these two techniques will therefore be provided. The first operates on a directed graph $G = (V, A)$ or on an undirected graph $G = (V, E)$, where $V = \{1, \ldots, n\}$ is a vertex set, $A = \{(i, j): i, j \in V, i \neq j\}$ is an arc set, and $E = \{(i, j): i, j \in V, i < j\}$ is an edge set. The computation of shortest paths with turn penalties is usually carried out on a directed graph. In this study it is assumed that G is rectilinear (ie all arcs or edges meet at 90° or 180° angles) so that the concept of right or left turn is well defined.

The Chinese postman problem in directed or undirected graphs

A graph is Eulerian or unicursal if there exists a closed traversal using all arcs or edges exactly once. Necessary and sufficient conditions for unicursality are stated by Ford and Fulkerson.[15] If G is directed, it must be strongly connected and the number of arcs entering each vertex must be equal to the number of arcs leaving it. If G is undirected, it must be connected and all its vertices must have an even degree. The CPP, introduced by Guan,[16] consists of determining a least cost closed traversal of a graph G. This is easily done in two steps.

Step 1. If G is Eulerian, proceed to Step 2. Otherwise, solve a least cost *augmentation problem* on G, ie replicate in a least cost manner some arcs or edges of G such that the resulting graph is Eulerian. This can be

accomplished through the solution of a transportation problem if G is directed, or of a matching problem if G is undirected.[17]

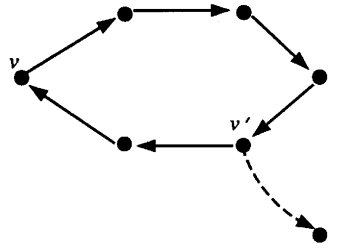
Step 2. Given that G is Eulerian, a closed traversal of all its arcs or edges can be determined by means of the following *end-pairing algorithm*.[17]

Step 2.1. Starting at an arbitrary vertex v traverse a yet untraversed arc or edge and repeat this operation until the starting vertex is reached.

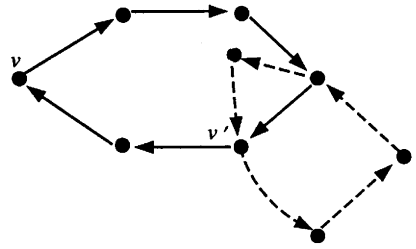
Step 2.2. If all arcs or edges have been traversed, stop. Otherwise select a vertex v' on the current circuit (if G is directed) or cycle (if G is undirected) incident to an untraversed arc or edge, and construct another circuit or cycle starting with v'.

Step 2.3. Merge the two circuits or cycles $(\ldots, i, v', j, \ldots)$ and $(\ldots, k, v', l, \ldots)$ into a single circuit or cycle $(\ldots, i, v', l, \ldots, k, v', j, \ldots)$ and go to Step 2.2.

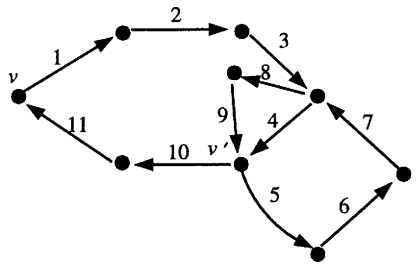
Steps 2.2 and 2.3 are illustrated in Figure 1.



a) Circuit obtained after Step 2.1 (full arcs), and first untraversed arc of the second circuit (dashed arc) incident to vertex v'.



b) Second circuit (dashed arc) obtained after Step 2.2.

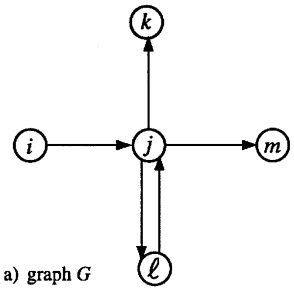


c) Merged circuit obtained after Step 2.3. The order of traversal is indicated by arc numbers.

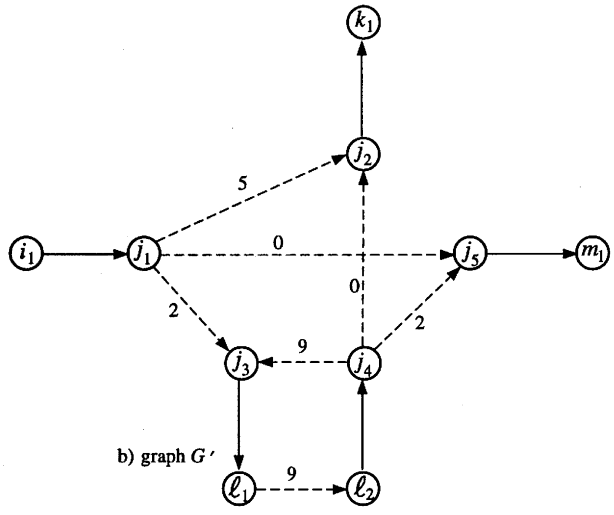
Figure 1 End-pairing algorithm.

Shortest paths with turn penalties in directed graphs

The first algorithms for the computation of shortest paths with turn penalties in a directed graph $G = (V, A)$ can be traced back to the work of Caldwell,[18] Kirby[19] and several other researchers. In a survey and appraisal of the early literature on this topic, Kirby and Potts[20] report that all proposed algorithms are equivalent. The underlying idea in these algorithms is to work on an auxiliary graph $G' = (V', A')$ constructed as follows. First, include in A' a 'loose' copy of every arc (i, j) of A, ie if two arcs are incident in A, they have no vertex in common in A'. These arcs are labelled (i_r, j_s), where i and j are the extremities of the corresponding arc in A. The indices r and s are required since the same vertex i or j of V may be incident to several arcs of A'. In addition, include in A' a penalty arc (j_s, j_t) wherever a turn is permitted from the arc corresponding to (i_r, j_s) to the arc corresponding to (j_t, k_u) in G, with the appropriate turn penalty. The vertex set V' is the set of all end vertices of arcs of A'. The construction of G' is illustrated in Figure 2 for the following turn penalties: 0 for a straight crossing, 2 for a right turn, 5 for a left turn, and 9 for a U-turn.



a) graph G



b) graph G'

Figure 2 Construction of G' from G. The copies of arcs of G are shown as full lines. The penalty arcs are shown as broken lines, with the corresponding turn penalty.

Using this construction, the shortest path with turn penalties between any two vertices of graph G is easily obtained by carrying out the corresponding shortest path computation on G'. Recent implementations of this approach are reported by Namkoong *et al*[21] and Bouroujerdi and Uhlmann.[22]

Transformation procedures for the Chinese postman problem with turn penalties

Two approaches for solving the CPP on a directed or on an undirected graph are now described. Both make use of a technique recently proposed by Laporte[23] and transform the original problem into an equivalent asymmetric TSP.

Transformation for directed graphs

Given a directed graph $G(V, A)$, construct a complete directed graph H whose vertices w_{ij} correspond to the arcs (i, j) of G. The cost c_{jl} of arc (w_{ij}, w_{kl}) on H is equal to the shortest path length in G' between the vertex corresponding to the end j of arc (i, j) and the vertex corresponding to the end l of arc (k, l). Note that this cost includes all turn penalties starting from the arc ending at j and finishing with the arc ending at l. This is illustrated in Figure 3 (the turn penalties are those of Figure 2). In all diagrams, the costs are indicated on the arcs.

Solving a CPP with turn penalties on G then amounts to solving an asymmetric TSP on H. Several algorithms have been proposed for the asymmetric TSP. One of the most efficient (and for which a computer code is readily available) is the branch-and-bound procedure (CT) described by Carpaneto and Toth.[24] This algorithm first solves an assignment problem on the vertices of H, using the costs c_{jk} just defined. The solution is either a circuit containing all vertices, in which case it constitutes a feasible and optimal solution, or a collection of disjoint circuits (subtours), in which case it is infeasible. A feasible and optimal TSP solution can be reached by branching on the arcs of subtours, as suggested by Carpaneto and Toth. When

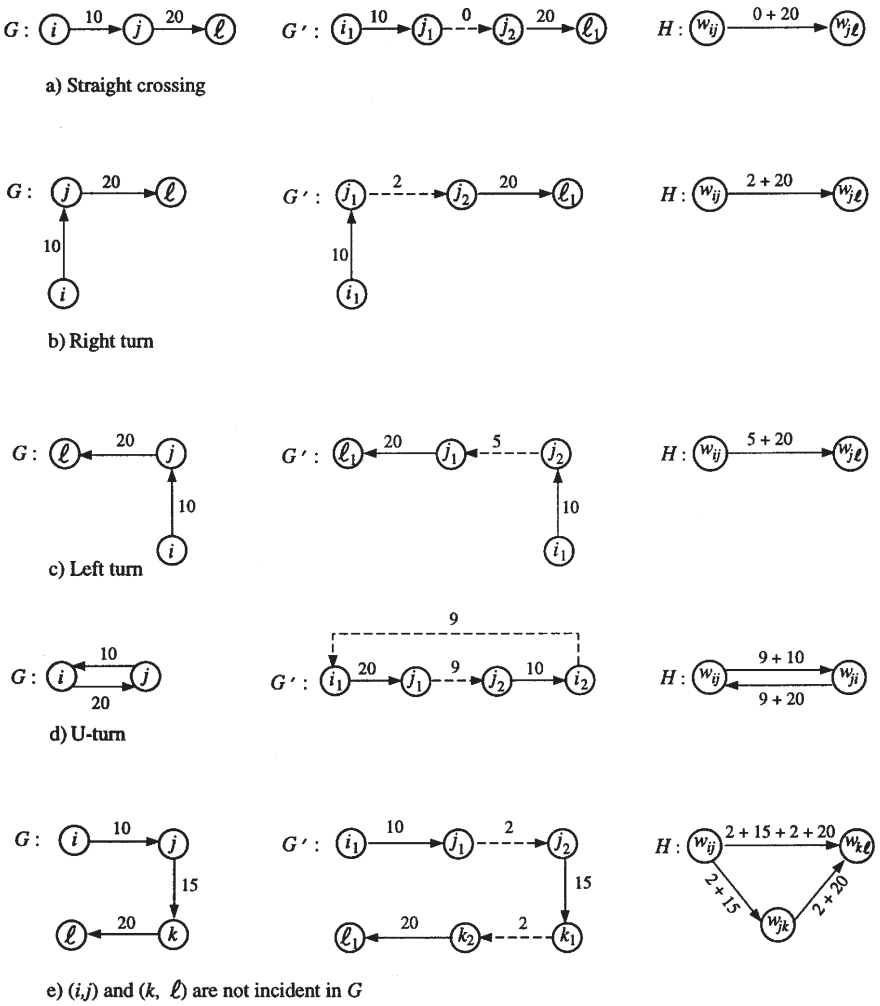


Figure 3 Computation of arc costs in H.

problem size becomes too large, the CT algorithm may fail to provide a solution within reasonable time or memory requirements may become excessive. In such cases, it is advisable to resort to a heuristic. Karp's *patching algorithm*[25] (PAT) is probably the best. It is at the same time fast and highly effective (it usually yields optimal or near-optimal solutions). It is also easy to implement since it only requires adding a simple routine to the assignment step of the CT algorithm. This heuristic first solves the assignment problem. If the solution consists of a single tour, it is then optimal. Otherwise, subtours are gradually merged into a single tour. The most common implementation of the PAT heuristic determines the two subtours containing the largest number of arcs and merges them in a least cost manner, ie it identifies an arc (p, q) from the first subtour and an arc (r, s) from the second subtour such that $(c_{ps} + c_{rq} - c_{pq} - c_{rs})$ is minimized. It then deletes arcs (p, q) and (r, s), and introduces arcs (r, p) and (p, s) to merge the two subtours. This operation is repeated until only one subtour remains. The procedure proposed by Bodin and Kursh,[2] and later applied by McBride,[6] works also along these lines but is not identical. It uses the same graph transformation and solves an assignment problem as in the first step of the Carpaneto and Toth algorithm,[24] but it then constructs a single tour by means of a shortest spanning tree computation. Benavent and Soler[26] have recently proposed two new constructive heuristics and one post-optimization procedure based on the same transformation.

Transformation for undirected graphs

If $G = (V, E)$ is an undirected graph, one can define a procedure inspired from the transformation proposed by Laporte[23] for several families of arc routing problems. First, construct from G a directed graph G' containing two opposite arcs (i, j) and (j, i) having the same cost for each edge (i, j) of G. In the CPP solution, one of these two arcs must be serviced, but both may be traversed if necessary. Then construct graph H from G' as in the directed case. The problem to be solved on H is no longer a TSP, but an asymmetric *Generalized Travelling Salesman Problem* (GTSP) which consists of determining a least cost Hamiltonian circuit visiting each of several clusters at least once. Here the clusters are of the form $\{w_{ij}, w_{ji}\}$. If only one of these two vertices is visited, this means edge $(i, j) \in E$ will be traversed in only one direction. If both are visited, then (i, j) will be traversed in both directions. The asymmetric GTSP on H just defined can now be transformed into an asymmetric TSP on an associated graph H', having the same number of vertices, by means of a procedure proposed by Noon and Bean.[27] First add a very large constant M to the cost of each intercluster arc. Then replace the cost of every arc (w_{ij}, w_{kl}) linking two clusters by the cost of arc (w_{ji}, w_{kl}). The solution to an asymmetric TSP on H' can then be interpreted as a solution to a CPP with turn penalties on G. The TSP solution will contain $|E|$ arcs of cost M, one per cluster. The CPP solution cost is therefore equal to the TSP cost, minus $M|E|$. As far as the authors are aware, this approach has never been tested. However, Roy and Rousseau[9] use a similar transformation of G into an auxiliary graph H' containing $3|E| + 1$ vertices (as opposed to $2|E|$ in our case), following a method proposed by Pearn,[28] and the resulting TSP is solved heuristically using a nearest-neighbour heuristic. They report no computational results.

A direct method for handling turn penalties

An alternative to the graph transformation approach just described is to solve the problem as a CPP and simply handle turn penalties directly within the end-pairing algorithm. This approach consists of first transforming G into a Eulerian graph and then applying the end-pairing algorithm by selecting in Step 2.1 the next arc or edge so as to keep turn penalties low. In all, six strategies were tested.

Strategy 1. From vertex v, successively follow the arc or edge (v, i), where i is the lowest vertex index. If G is undirected, follow the final cycle in both directions and select the better orientation, taking turning costs into account.

Strategy 2. From vertex v, follow the arc or edge (v, i) yielding the least turn penalty. In undirected graphs, when merging two cycles meeting at vertex v, consider the best combined orientation of the two cycles taking turn costs into account. This rule also applies to strategies 2, 4, 5 and 6.

Strategy 3. This is a look-ahead procedure. Whenever a new arc or edge (v, i) is selected, the choice of the next arc or edge (i, j) is also considered (if any is available) and the path or chain (v, i, j) is added to the current circuit or cycle.

Strategy 4. This strategy is similar to strategy 3 except that, after having identified the best sequence (v, i, j), only (v, i) is added to the current circuit or cycle.

Strategy 5. From vertex v, select (v, i) such that i has a minimum residual degree (ie the least number of incident arcs or edges not yet selected).

Strategy 6. From vertex v, select (v, i) such that i has a maximum residual degree.

The interest of this direct approach is that it avoids transforming G into a larger graph and it does not require the solution of an assignment problem. It remains to be seen how it performs in terms of solution quality and computing time.

Computational results

The graph transformation procedures and heuristics described in the previous sections were coded in C. For the solution of the TSP's, a C version of the Carpaneto and

Toth algorithm,[24] automatically translated from FORTRAN, was used. All computational tests were executed on a Sun Ultra-10 station. The graph generation procedure and the actual computational results are also described.

Graph generation

All tests were carried out on undirected graphs which represent a greater challenge than directed graphs[23] and for which no previous computational experience is available. Undirected graphs were generated on rectangular grids corresponding to the integer lattice $[0, \alpha] \times [0, \beta]$. Only a given proportion λ of all potential $\alpha(\beta + 1) + \beta(\alpha + 1)$ edges were generated. To produce these graphs we repeatedly applied the following procedure.

Step 1. Declare all vertices of the grid inactive, except only one. There is yet no created edge.

Step 2. Randomly select an active vertex v. Randomly select a direction (out of 2, 3 or 4) in which it is possible to move from v without going outside the grid and without moving on an already created edge. Moving in this direction, compute μ as the minimum of the following two distances: (i) from v to the beginning of the first encountered created edge, (ii) from v to the grid boundary. From v, create an edge (v, v') of integer length (and cost) θ, randomly selected in the interval $[1, \mu]$. Declare v' active. Repeat this step until the required proportion λ of edges have been created.

Step 3. Coalesce into one all parallel edges meeting at vertices at which no other edge is incident.

Table 1 Computational comparison of the six edge selection strategies in the end-pairing algorithm

		Strategy 1		*Strategy 2*		*Strategy 3*		*Strategy 4*		*Strategy 5*		*Strategy 6*	
$\alpha \times \beta$	*Edge density* λ	*Cost*	*Seconds*	*Cost*	*Seconds*	*Cost*	*Seconds*	*Cost*	*Seconds*	*Cost*	*Seconds*	*Cost*	*Seconds*
2×2	0.20	**19.6**	0.00	**19.6**	0.00	**19.6**	0.00	**19.6**	0.00	**19.6**	0.00	21.6	0.00
	0.40	**26.7**	0.00	27.2	0.00	27.2	0.00	27.2	0.00	**26.7**	0.00	27.2	0.00
	0.60	**29.3**	0.00	29.9	0.00	29.7	0.00	30.2	0.00	29.9	0.00	29.8	0.00
	0.80	31.5	0.00	29.6	0.00	**29.0**	0.00	**29.0**	0.00	33.2	0.00	30.4	0.00
3×3	0.20	**24.4**	0.00	24.9	0.00	24.9	0.00	24.9	0.00	**24.4**	0.00	26.9	0.00
	0.40	43.0	0.00	**40.1**	0.00	42.1	0.00	43.0	0.00	44.2	0.00	42.8	0.00
	0.60	62.2	0.00	58.0	0.00	58.8	0.00	59.1	0.00	63.2	0.00	**57.9**	0.00
	0.80	72.9	0.00	60.1	0.00	57.5	0.00	**52.7**	0.00	72.9	0.00	63.9	0.00
3×4	0.20	**20.2**	0.00	29.7	0.00	29.7	0.00	29.7	0.00	30.7	0.00	29.7	0.00
	0.40	44.7	0.00	44.6	0.00	**41.9**	0.00	43.8	0.00	44.0	0.00	**41.9**	0.00
	0.60	72.0	0.00	**63.1**	0.00	67.2	0.00	65.3	0.00	73.1	0.00	63.6	0.00
10×10	0.20	146.3	0.00	**124.6**	0.00	134.1	0.00	149.5	0.00	147.8	0.00	143.4	0.00
	0.40	311.9	0.00	272.5	0.00	273.8	0.00	**268.9**	0.00	306.4	0.00	286.4	0.00
	0.60	473.4	0.00	421.6	0.00	376.9	0.00	**369.3**	0.01	481.8	0.00	431.7	0.00
	0.80	567.6	0.01	503.5	0.00	429.1	0.00	**402.5**	0.01	571.2	0.00	494.6	0.00
20×20	0.20	404.7	0.00	**351.5**	0.00	365.6	0.00	377.3	0.00	407.3	0.00	380.4	0.00
	0.40	879.0	0.01	765.2	0.01	**700.0**	0.00	701.9	0.01	886.7	0.00	806.4	0.00
	0.60	1485.6	0.03	1325.7	0.01	1100.5	0.01	**1029.8**	0.01	1520.2	0.01	1295.3	0.01
	0.80	2004.9	0.07	1827.8	0.01	1337.4	0.01	**1219.2**	0.02	2058.6	0.01	1768.2	0.01
30×30	0.20	755.8	0.01	**645.1**	0.00	658.8	0.01	670.8	0.01	783.8	0.00	708.8	0.00
	0.40	1767.6	0.04	1578.4	0.01	1336.1	0.01	**1332.5**	0.02	1834.7	0.01	1639.9	0.01
	0.60	2995.0	0.08	2725.3	0.01	2112.8	0.01	**1991.6**	0.02	3145.1	0.02	2682.8	0.02
	0.80	4247.8	0.18	3863.9	0.01	2670.9	0.02	**2459.1**	0.03	4353.8	0.02	3679.6	0.02
10×30	0.20	370.5	0.00	**319.9**	0.00	334.9	0.01	348.9	0.00	378.2	0.00	360.4	0.00
	0.40	803.8	0.01	676.5	0.00	**645.7**	0.01	665.5	0.00	801.0	0.00	718.1	0.00
	0.60	1153.9	0.02	1038.2	0.01	**888.4**	0.01	888.8	0.01	1209.7	0.01	1048.5	0.01
	0.80	1543.8	0.04	1426.9	0.00	1076.2	0.01	**1029.8**	0.02	1609.3	0.01	1380.2	0.01
10×50	0.20	543.1	0.01	**471.0**	0.00	496.0	0.00	511.4	0.01	542.2	0.00	522.7	0.00
	0.40	1139.3	0.01	981.1	0.01	**907.0**	0.01	931.1	0.01	1181.3	0.01	1038.8	0.01
	0.60	1742.2	0.05	1555.5	0.01	1344.9	0.01	**1293.8**	0.02	1811.8	0.01	1549.0	0.01
	0.80	2455.8	0.11	2268.9	0.01	1731.8	0.02	**1600.4**	0.03	2593.4	0.02	2159.3	0.02
20×50	0.20	868.0	0.01	742.1	0.01	**735.4**	0.00	756.3	0.01	899.4	0.01	838.9	0.01
	0.40	2025.1	0.06	1776.7	0.01	1543.9	0.02	**1521.4**	0.02	2149.2	0.02	1852.4	0.02
	0.60	3321.0	0.18	2948.4	0.02	2330.6	0.03	**2217.4**	0.04	3440.6	0.03	2936.2	0.03
	0.80	4575.4	0.24	4274.4	0.01	3002.7	0.02	**2705.0**	0.04	4786.6	0.03	3973.4	0.03

This way of proceeding always produces undirected connected graphs since every newly created edge originates from an active vertex.

Test results and analyses

Comparisons of the six strategies just described with two other approaches were carried out by running tests on graphs of various sizes. The two algorithms used in the comparisons are the application of the Carpaneto and Toth[24] exact algorithm (CT) to a transformed graph, and the heuristic (PAT) obtained by first solving an assignment problem on the transformed graph and then patching subtours together. In these tests the following penalties were used: straight crossing: 2, right turn: 0, left turn: 5, U-turn: 9. These penalties are representative of those used in snow clearing operations where it is often more costly to cross a street than to turn right,[13] since straight crossings leave a trail of snow across the intersection whereas right-turns avoid this situation. The solution cost and the computation time in seconds for each of the six strategies are reported in Table 1, with the best cost values shown in bold characters. All reported statistics are average values on ten different randomly generated instances. These results reveal that all computing times are insignificant and the best strategies are 2 for low densities (0.20) and 4 for higher densities (> 0.20).

Table 2 compares strategies 2 and 4 with CT and PAT. Denote by z_2, z_4, z_{CT} and z_{PAT} the solution values corresponding to these four algorithms. Since strategies 2 and 4 as well as PAT are heuristics, it follows that $z_2/z_{CT} \geq 1$ and $z_4/z_{CT} \geq 1$. Of interest are the size of these ratios and the values of z_2/z_{PAT} and z_4/z_{PAT}. Table 2 also provides the computing time of each algorithm. Missing values correspond to cases where memory or time requirements prevented the completion of the algorithm.

Results presented in Table 2 indicate that for graphs having a low edge density, both strategies 2 and 4 provide

Table 2 Comparison of four algorithms

		Carpaneto and Toth		*Patching*		*Strategy 2*				*Strategy 4*			
$\alpha \times \beta$	*Edge density* λ	z_{CT}	*Seconds*	z_{PAT}	*Seconds*	z_2	z_2/z_{CT}	z_2/z_{PAT}	*Seconds*	z_4	z_4/z_{CT}	z_4/z_{PAT}	*Seconds*
2×2	0.20	19.6	0.01	22.6	0.00	19.6	1.00	0.87	0.00	19.6	1.00	0.87	0.00
	0.40	26.7	0.01	29.2	0.00	27.2	1.02	0.93	0.00	27.2	1.02	0.93	0.00
	0.60	29.2	0.02	36.4	0.01	29.9	1.02	0.82	0.00	30.2	1.03	0.83	0.00
	0.80	22.8	0.03	37.7	0.01	29.6	1.30	0.79	0.00	29.0	1.27	0.77	0.00
3×3	0.20	24.4	0.01	25.9	0.00	24.9	1.02	0.96	0.00	24.9	1.02	0.96	0.00
	0.40	40.0	0.08	40.6	0.01	40.1	1.00	0.99	0.00	43.0	1.07	1.06	0.00
	0.60	50.6	0.90	64.5	0.03	58.0	1.15	0.90	0.00	59.1	1.17	0.92	0.00
	0.80	41.5	9.64	75.0	0.06	60.1	1.45	0.80	0.00	52.7	1.27	0.70	0.00
3×4	0.20	28.7	0.01	31.2	0.01	29.7	1.03	0.95	0.00	29.7	1.03	0.95	0.00
	0.40	37.6	0.08	49.1	0.02	44.6	1.19	0.91	0.00	43.8	1.15	0.88	0.00
	0.60	50.4	3.22	67.9	0.05	63.1	1.25	0.93	0.00	65.3	1.30	0.96	0.00
10×10	0.20	—	—	128.4	0.42	124.6	—	0.97	0.00	149.5	—	1.16	0.00
	0.40	—	—	278.4	5.54	272.5	—	0.98	0.00	268.9	—	0.97	0.00
	0.60	—	—	428.7	35.24	421.6	—	0.98	0.00	369.3	—	0.86	0.01
	0.80	—	—	541.1	101.82	503.5	—	0.93	0.00	402.5	—	0.74	0.01
20×20	0.02	—	—	357.6	12.82	351.5	—	0.98	0.00	377.3	—	1.06	0.00
	0.40	—	—	—	—	765.2	—	—	0.01	701.9	—	—	0.01
	0.60	—	—	—	—	1325.7	—	—	0.01	1029.8	—	—	0.01
	0.80	—	—	—	—	1827.8	—	—	0.01	1219.2	—	—	0.02
30×30	0.20	—	—	—	—	645.1	—	—	0.00	670.8	—	—	0.01
	0.40	—	—	—	—	1578.4	—	—	0.01	1332.5	—	—	0.02
	0.60	—	—	—	—	2725.3	—	—	0.01	1991.6	—	—	0.02
	0.80	—	—	—	—	3863.9	—	—	0.01	2459.1	—	—	0.03
10×30	0.20	—	—	326.4	9.24	319.9	—	0.98	0.00	348.9	—	1.07	0.00
	0.40	—	—	—	—	676.5	—	—	0.00	665.5	—	—	0.00
	0.60	—	—	—	—	1038.2	—	—	0.01	888.8	—	—	0.01
	0.80	—	—	—	—	1426.9	—	—	0.00	1029.8	—	—	0.02
10×50	0.20	—	—	482.9	37.29	471.0	—	0.98	0.00	511.4	—	1.06	0.01
	0.40	—	—	—	—	981.1	—	—	0.01	931.1	—	—	0.01
	0.60	—	—	—	—	1555.5	—	—	0.01	1293.8	—	—	0.02
	0.80	—	—	—	—	2268.9	—	—	0.01	1600.4	—	—	0.03
20×50	0.20	—	—	—	—	742.1	—	—	0.01	756.3	—	—	0.01
	0.40	—	—	—	—	1776.7	—	—	0.01	1521.4	—	—	0.02
	0.60	—	—	—	—	2948.4	—	—	0.02	2217.4	—	—	0.04
	0.80	—	—	—	—	4274.4	—	—	0.01	2705.0	—	—	0.04

solution values that are within a few percentage points of the optimum (as indicated by the ratios z_2/z_{CT} and z_4/z_{CT}), but there is a significant departure from optimality when the graph density increases. The two statistics z_2/z_{PAT} and z_4/z_{PAT} are less than 1 in most cases, indicating that strategies 2 and 4 are better heuristics than the assignment algorithm followed by subtour patching. In terms of computing times, strategies 2 and 4 clearly outperform the other two algorithms. Since only very small instances of our problem can be solved exactly by means of the Carpaneto and Toth algorithm,[24] the most satisfactory heuristic for handling turn penalties seems to be the use of an end-pairing algorithm on the initial graph G, in conjunction with strategy 2 or 4. In closing it is worth mentioning that tests were also performed with several other settings of turn penalties and identical conclusions were reached.

Conclusion

Arc routing problems with turn penalties arise in a number of practical settings. Yet they are very difficult to solve to optimality and the use of heuristics is the only realistic approach. All previous methods rely on a transformation of the problem into an equivalent TSP. Tests show that only relatively small instances can be solved optimally using this approach. This study shows, not surprisingly, that using a direct approach is much faster than solving a TSP exactly. It is also superior in terms of time and solution quality than using the patching heuristic, regarded as one of the best for the asymmetric TSP (it typically produces optimal or near-optimal solutions). In addition, the computation time required by the direct approach is relatively low.

Acknowledgement—This research was in part supported by the Canadian Natural Sciences and Engineering Research Council under grants OPG0039682 and OPG0184121, and by Quebec FCAR grants ER0289 and EN0020. The authors are grateful to Nathalie Perrier who provided some useful references, and to Giorgio Carpaneto and Paolo Toth for making their TSP code available. Thanks are also due to the referees for their valuable comments.

References

1 Haslam E and Wright JR (1991). Applications of routing technologies to rural snow and ice control. *Transport Res Rec* **1304**: 202–211.
2 Bodin LD and Kursh SJ (1979). A detailed description of a computer system for the routing and scheduling of street sweepers. *Comput Oper Res* **6**: 181–198.
3 Eglese RW (1994). Routing winter gritting vehicles. *Discrete Applied Mathematics* **48**: 231–244.
4 Eglese RW and Li LYO (1992). Efficient routing for winter gritting. *J Opl Res Soc* **43**: 1031–1034.
5 Eglese RW and Murdock H (1991). Routing road sweepers in a rural area. *J Opl Res Soc* **42**: 281–288.
6 McBride R (1982). Controlling left and U-turns in the routing of refuse collection vehicles. *Comput Oper Res* **9**: 145–152.
7 Alvarez-Valdes R, Benavent E, Campos V, Corberán A, Mota E, Tamarit JM and Valls V (1993). ARC, a computerised system for urban garbage collection. Working paper, Departamentó de Estadoística e Investigación Operativa, Universidad de Valencia, Spain.
8 Walsh B, Bodin LD, Levy L and Fagan G (1994). The United States postal service Geomod system. *Proceedings of the USPS Advanced Technology Conference*, United States Postal Services.
9 Roy S and Rousseau J-M (1988). The capacitated Canadian postman problem. *INFOR* **27**: 58–73.
10 Wunderlich J, Collette M, Levy L and Bodin LD (1992). Scheduling meter readers for Southern California Gas Company. *Interfaces* **22**(3): 22–30.
11 Eiselt HA, Gendreau M and Laporte G (1995). Arc routing problems, part II: The rural postman problem. *Opns Res* **43**: 399–414.
12 Assad AA and Golden BL (1995). Arc routing methods and applications. In: *Network Routing, Handbooks in Operations Research and Management Science*, Vol. 8, Ball MO, Magnanti TL, Monma CL and Nemhauser GL (eds). North-Holland: Amsterdam, pp 375–483.
13 Gendreau M, Laporte G and Yelle SY (1997). Efficient routing of service vehicles. *Engineering Optimization* **28**: 263–271.
14 Lee JH, Chung C-Y, Lee BH and Choi MH (1998). Turn penalty problem model for minimum time AGV routing. *Proceedings of the IECON'97 23rd International Conference on Industrial Electronics, Control and Instrumentation*, Vol 3, pp 1323–1326.
15 Ford LR and Fulkerson DR (1962). *Flows in Networks*. Princeton University Press: Princeton, NJ.
16 Guan M (1962). Graphic programming using odd and even points. *Chinese Math* **1**: 273–277.
17 Edmonds J and Johnson EL (1973). Matching, Euler tours and the Chinese postman problem. *Math Prog* **5**: 88–124.
18 Caldwell T (1961). On finding minimum routes in a network with turn penalties. *Commun ACM* **4**: 107–108.
19 Kirby RF (1966). A minimum path algorithm for a road network with turn penalties. *Proceedings of the 3rd Australian Road Research Board Conference*, Part 1, pp 434–442.
20 Kirby RF and Potts RB (1969). The minimum route problem for networks with turn penalties and prohibitions. *Transport Res* **3**: 397–408.
21 Namkoong S, Rho JH and Choi J-U (1998). Development of the tree-based link labelling algorithm for optimal path-finding in urban transportation networks. *Math Comput Model* **27**: 51–65.
22 Bouroujerdi A and Uhlmann J (1998). An efficient algorithm for computing least cost path with turn constraints. *Info Process Lett* **67**: 317–321.
23 Laporte G (1997). Modeling and solving several classes of arc routing problems as traveling salesman problems. *Comput Oper Res* **24**: 1057–1061.
24 Carpaneto G and Toth P (1980). Some new branching and bounding criteria for the asymmetric traveling salesman problem. *Mgmt Sci* **26**: 736–743.
25 Karp RM (1979). A patching algorithm for the nonsymmetric traveling-salesman problem. *SIAM J Comput* **8**: 561–573.
26 Benavent E and Soler D (1999). The directed rural postman problem with turn penalties. *Transport Sci* **33**: 408–418.
27 Noon CE and Bean JC (1993). An efficient transformation of the generalized traveling salesman problem. *INFOR* **31**: 39–44.
28 Pearn WL (1984). The capacitated Chinese postman problem. PhD dissertation, University of Maryland at College Park.

Received September 1999;
accepted August 2000 after two revisions